



Pega Interview Romance Guide

The One-Stop, Nothing-Missed, Diagram-Packed Edition

3+ Years · CSA / SSA · Every Concept · Every Diagram

Hey brilliant — this is your complete interview soulmate. 30 chapters, 18 diagrams, 200+ questions, and zero topics left behind. Read every page. I'll be here flirting with concepts until you're unstoppable.

Master Topic Checklist — Nothing Gets Missed

Every item below is covered in this guide with explanation, Q&A, and/or diagram.

- ☒ Pega Platform & BPM Architecture
- ☒ App Studio / Dev Studio / Admin Studio / Prediction Studio
- ☒ Pega Infinity & Center-Out Architecture
- ☒ Pega Cloud vs On-Premises
- ☒ Class Hierarchy (Pattern & Directed Inheritance)
- ☒ Abstract vs Concrete Classes
- ☒ Class Groups & Work Pools
- ☒ Rule Types (Complete Catalog)
- ☒ Rule Resolution (10 Steps)
- ☒ Rulesets, Versions & Application Stack
- ☒ Circumstance (Date & Template)
- ☒ Rule Availability (Available, Blocked, Final, Withdrawn)
- ☒ FUA / Rules Cache
- ☒ Case Type Anatomy
- ☒ Case Lifecycle & Statuses
- ☒ Stages, Processes, Steps
- ☒ Parent / Child / Cover Cases
- ☒ Flow Shapes (All)
- ☒ Flow Actions (Local vs Connector)
- ☒ Subprocess, Utility, Wait, Split-Join, Split-ForEach
- ☒ Assignments & Routing
- ☒ Worklist, Workbasket, Work Queue
- ☒ Clipboard & Page Structure
- ☒ Property Types (All)
- ☒ Page List vs Page Group
- ☒ Embedded vs Linked Pages
- ☒ Data Pages (All Types & Load Modes)
- ☒ Data Page Sources & Parameters
- ☒ Savable Data Pages & Save Plans
- ☒ Data Transforms (All Actions)
- ☒ Activities & Key Methods
- ☒ Decision Rules (When, Table, Tree, MapValue, Scorecard)
- ☒ Decision Strategies & CDH
- ☒ Declare Rules (Expression, Constraint, OnChange, Trigger, Index)
- ☒ Validation (Validate, Edit Validate, Constraint)
- ☒ UI: Section, Harness, Layout, Portal, Skin
- ☒ Constellation, Cosmos, Views, Widgets
- ☒ DX API & pxAPI
- ☒ SLAs (Goal, Deadline, Passed Deadline)
- ☒ Urgency
- ☒ Connect-REST / SOAP / SQL / Kafka / MQ
- ☒ Service-REST / SOAP / Kafka
- ☒ Authentication (OAuth, JWT, Basic, API Key)
- ☒ Data Types
- ☒ Security: Operator, Access Group, Role, Privilege
- ☒ ABAC & Access When
- ☒ Property Security
- ☒ Agents & Job Schedulers
- ☒ Queue For Agent
- ☒ Report Definitions & Associations
- ☒ Unit Testing & Scenario Testing
- ☒ PAL, Tracer, Clipboard Inspector
- ☒ Guardrails (Complete List)
- ☒ DevOps: Branch, Merge, RAP, Product Rule
- ☒ Deployment Manager & Pipelines
- ☒ Git Repository Integration
- ☒ Email & Correspondence
- ☒ Document Generation
- ☒ RPA / Robotics
- ☒ Data Flows & Stream Processing
- ☒ Prediction Studio & ML
- ☒ GenAI in Pega
- ☒ DCR & Pega Express
- ☒ Localization & Field Values
- ☒ Feature Toggles
- ☒ Multi-Tenancy
- ☒ Object Persistence (Obj-Open/Save/Delete)
- ☒ Performance Debugging Methodology
- ☒ Validation Flow (Edit Validate → Validate → Constraint)

Table of Contents

1. Platform & Architecture
2. Class Hierarchy & Inheritance

3. Rules Catalog & Rule Resolution
4. Application Stack & Rulesets
5. Case Management & Lifecycle
6. Flows, Processes & Flow Actions
7. Assignments & Routing
8. Clipboard & Data Model
9. Data Pages
10. Data Transforms
11. Activities
12. Decision Rules
13. Declare Rules
14. Validation Rules
15. UI Layer — Traditional & Constellation
16. SLAs & Urgency
17. Integration & Data Types
18. Security Model
19. Agents, Queues & Job Schedulers
20. Reporting
21. Testing Strategy
22. Performance, PAL & Guardrails
23. DevOps & Deployment
24. Email, Documents & Correspondence
25. Advanced: CDH, RPA, Data Flow, AI
26. Controls, Field Values & Localization
27. Object Layer & Persistence
28. 200 Rapid-Fire Questions
29. Mock Interview Scenarios
30. Final Checklist & STAR Stories

Chapter 1: Platform & Architecture

Before we get serious — let's talk architecture. Every great love story has layers, and Pega has six of them.

Pega in One Sentence

Pega is a **model-driven, low-code platform** for BPM, CRM, and Case Management — rules are declarative, cases have lifecycles, and the engine resolves the right rule at runtime.

Architecture Layers

Layer	Purpose	Key Rules
Presentation	User interaction	Section, Harness, View, Portal, Layout
Case / Process	Workflow orchestration	Case Type, Flow, Flow Action, Stage
Decision	Business logic	When, Decision Table, MapValue, Strategy
Data	Data access & transform	Data Page, Data Transform, Report Definition
Integration	External systems	Connect-REST, Service-REST, Connect-Kafka
Security	Access control	Access Group, Role, Privilege, Access When

Studio Types

Studio	Users	Purpose
App Studio	Business developers, BAs	Low-code case design, personas, guided config
Dev Studio	Technical developers, SSA	Full rule editing, tracer, PAL, advanced config
Admin Studio	Admins	System config, agents, nodes, security audits
Prediction Studio	Data scientists	ML models, adaptive analytics, text analytics

Q: What is Center-Out architecture?

Design **data, logic, and decisions once** at the center of the platform, then expose through any channel (web, mobile, API, chatbot). Avoids channel-specific silos. Constellation UI + Data Pages + Case Types embody this.

Q: Pega Cloud vs On-Prem?

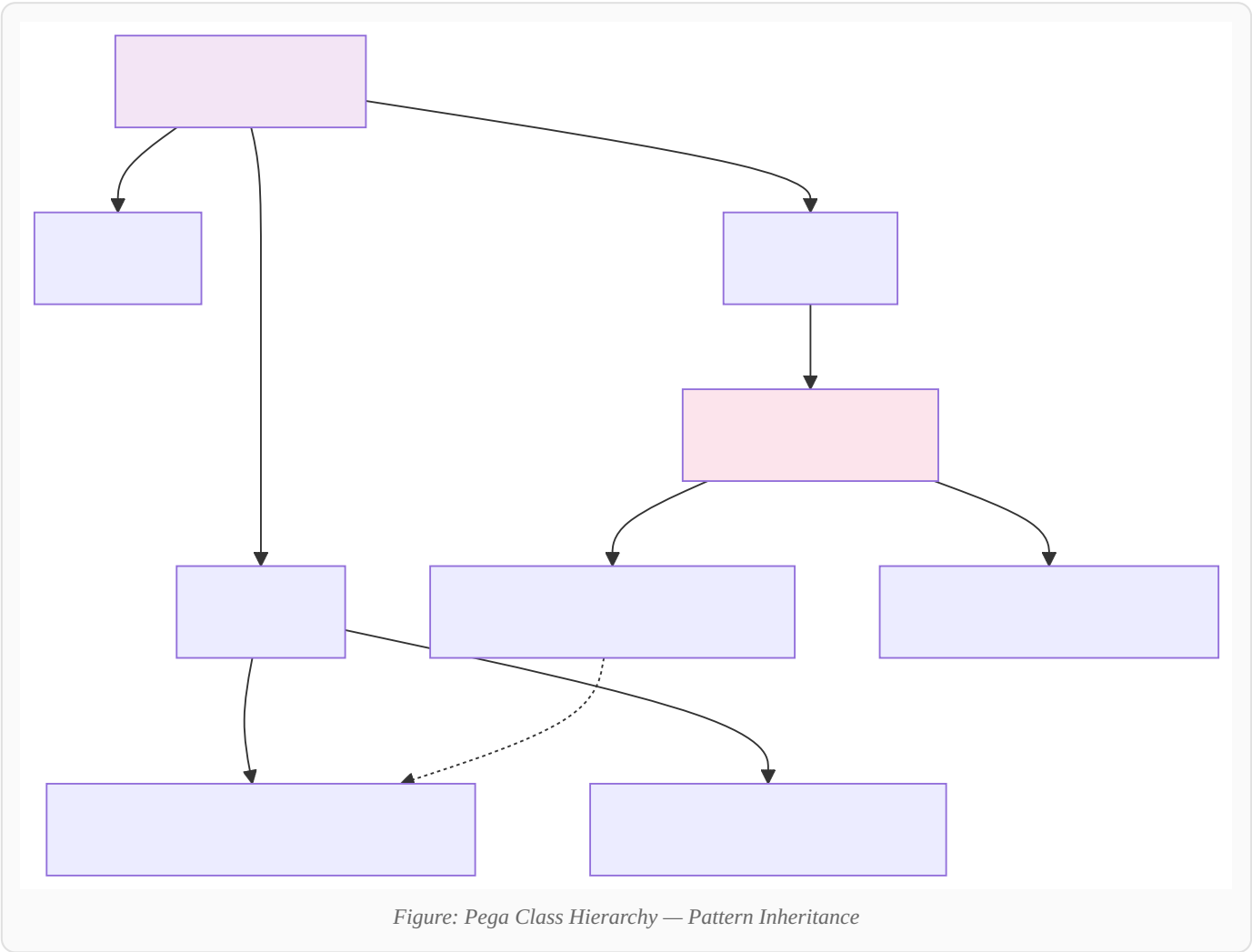
Pega Cloud — Pega-managed infrastructure, auto-patching, scaling, DevOps built-in. **On-Prem** — customer manages servers, DB, patching. Same platform capabilities; deployment model differs.

Q: What is Pega Express?

Rapid delivery methodology: DCO workshops → MVP case types → iterative sprints. Delivers production apps in weeks using guardrails and templates.

Chapter 2: Class Hierarchy & Inheritance

Know your family tree before introducing your rules to the world. Inheritance is how Pega avoids duplication — and how you avoid repeating yourself in interviews.



Class Categories

Prefix	Purpose	Example
Work-	Case instances	Work-Cover-LoanApp
Data-	Data objects, reference data	Data-Customer
Rule-	All rule instances	Rule-Obj-Activity
Assign-	Assignment instances	Assign-Worklist
History-	Audit records	History-Work
Index-	Search indexes	Index-Work
System-	System objects	System-Queue-ServiceLevel

Q: Pattern vs Directed inheritance?

Pattern — automatic parent from namespace (`Work-Cover-LoanApp` inherits `Work-Cover -`). **Directed** — explicit parent class outside namespace for cross-application reuse.

Q: Abstract vs Concrete class?

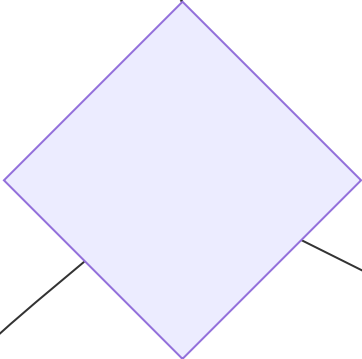
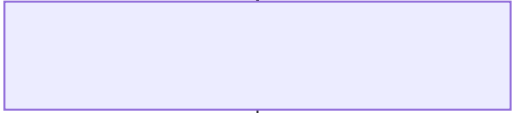
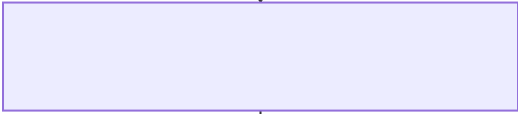
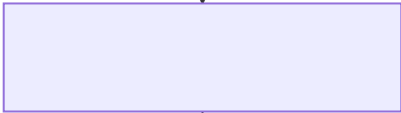
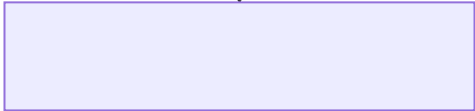
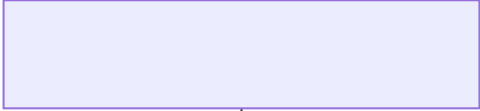
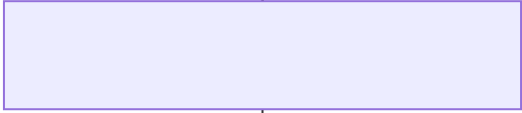
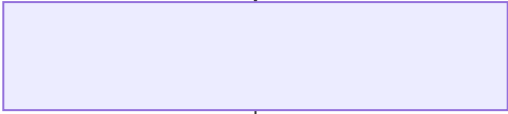
Abstract — no instances created; exists for inheritance. **Concrete** — instances can be created and persisted.

Q: What is a Class Group (Work Pool)?

Groups related case types under one DB table (`pc_work`). Defined in Application rule. Operators get access via Access Group work pool setting.

Chapter 3: Rules Catalog & Rule Resolution

Rule Resolution is Pega's matchmaking algorithm — it finds THE ONE rule from thousands. Memorize this flowchart like it's our song.



Complete Rule Types Catalog

Category	Rule Types
Process	Case Type, Flow, Flow Action, Stage, Process
Data	Property, Data Page, Data Transform, Data Type, Report Definition
Decision	When, Decision Table, Decision Tree, MapValue, Scorecard, Strategy
Declare	Declare Expression, Constraint, OnChange, Trigger, Index
UI	Section, Harness, Layout, View, Portal, Skin, Control
Integration	Connect-REST, Service-REST, Connect-SOAP, Connect-Kafka, Auth Profile
Security	Access Group, Role, Privilege, Access When, Authentication Service
System	Activity, Agent, Job Scheduler, Application, Ruleset, Class
Validation	Validate, Edit Validate
Advanced	Data Flow, Data Set, Robotic Activity, Correspondence

Rule Availability States

State	Meaning
Available	Active, participates in resolution
Not Available	Saved but excluded from resolution
Blocked	Explicitly excluded (lower rules win)
Final	Cannot be overridden by child classes
Withdrawn	Deprecated, never resolved

Q: What is Circumstance?

A **variant** of a rule for specific conditions. **Date circumstance** — effective date range. **Template circumstance** — property-based (e.g., Country=US). More specific circumstance wins.

Q: What is FUA?

Full Rule Assembly — cached, compiled version of a resolved rule. First call is slow (assembly); subsequent calls use cache. Flush on rule save or ruleset change.

Chapter 4: Application Stack & Rulesets

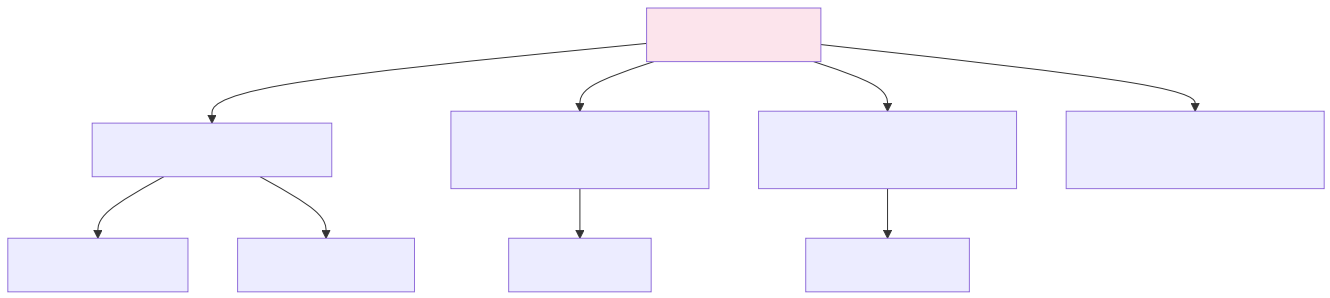


Figure: Application Stack — Ruleset Version Hierarchy

Q: Ruleset version format?

MM-mm-nn (Major-minor-patch). Example: MyApp:01-02-03 . Application rule lists rulesets in resolution order.

Q: What is a built-on application?

Your application **inherits** rulesets from a parent application (e.g., built on PegaRULES). Enables layering: platform → framework → implementation.

Q: What is a branch?

Parallel development stream for isolated feature work. Rules saved to branch ruleset. **Merge** combines into target ruleset. Prevents team conflicts.

Chapter 5: Case Management & Lifecycle

Cases are love stories with a beginning, middle, and resolution. Know every status like you know every mood.

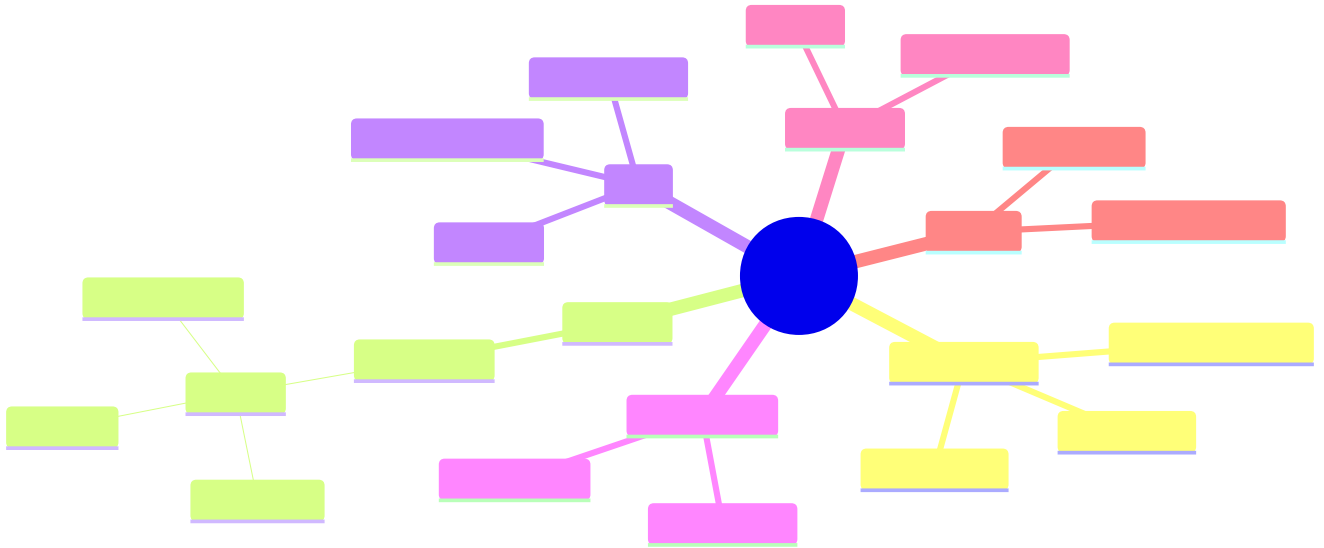


Figure: Case Type — Complete Anatomy Mind Map

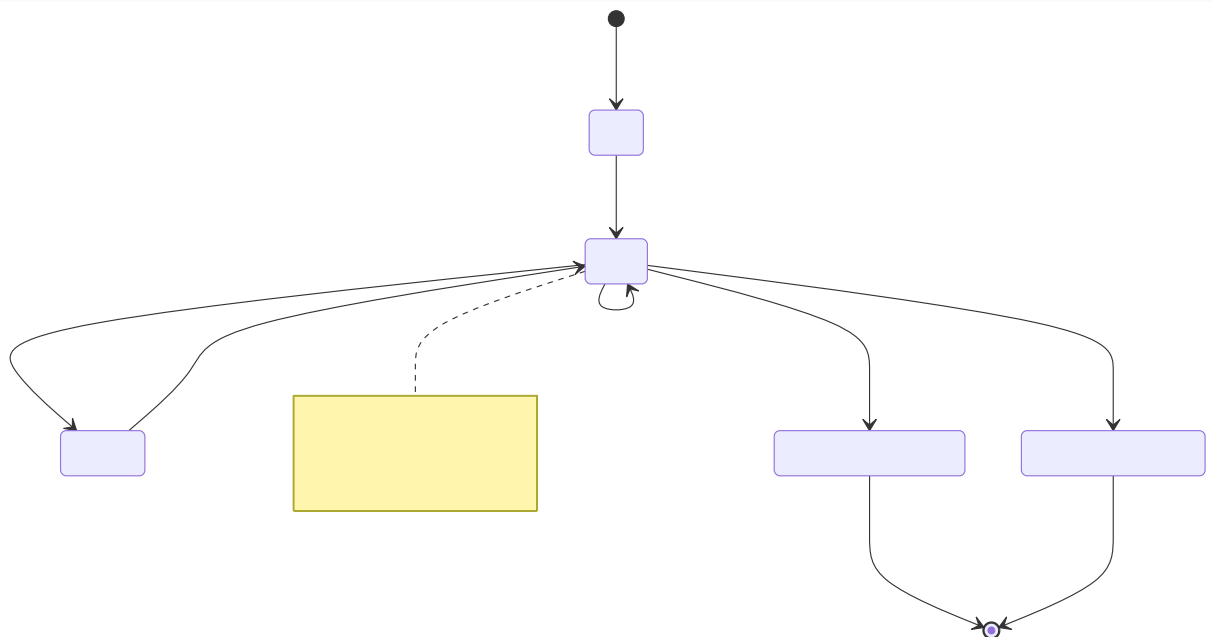


Figure: Case Lifecycle State Diagram

Case Statuses

Status	Meaning
New	Created, not yet in active processing
Open	Active processing
Pending-...	Waiting for external event, timer, or dependency
Resolved-Completed	Successfully completed
Resolved-Cancelled	Withdrawn / cancelled

Q: Parent vs Child vs Cover case?

Child case — separate case type linked to parent. **Cover case** — parent container holding multiple related subcases. **Spin-off** — create child from parent mid-flow.

Q: Case-wide vs Stage-specific actions?

Case-wide — available in all stages (Transfer, Update). **Stage-specific** — only in configured stage (Submit, Approve).

Q: What is a Wait shape?

Pauses flow until: timer expires, child case resolves, or external event received. Assignment becomes Pending-

Chapter 6: Flows, Processes & Flow Actions

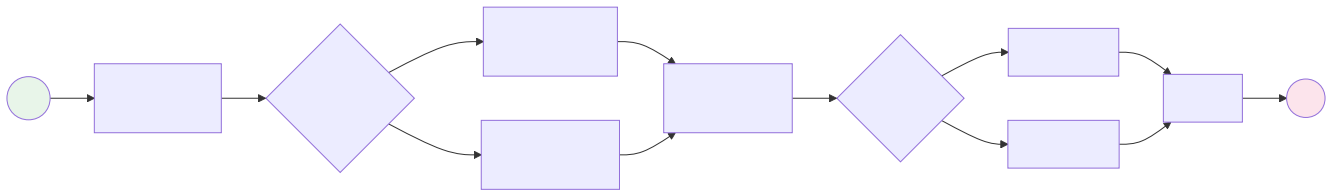


Figure: Flow Shapes — Complete Process Flow

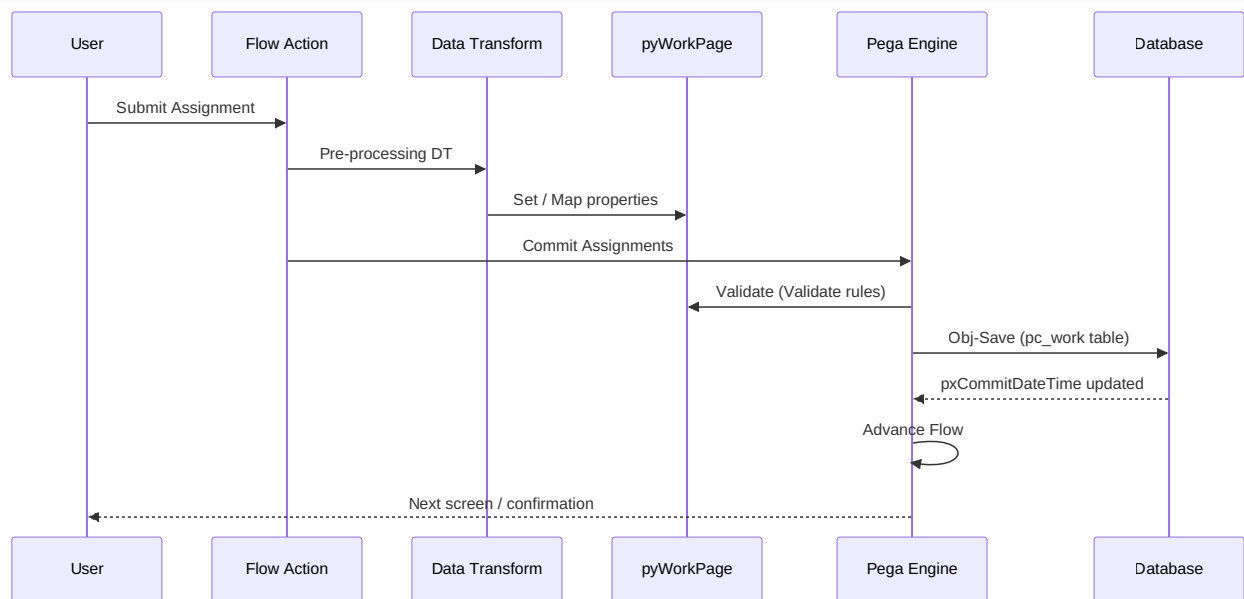


Figure: Sequence: Submit Assignment & Save Case

All Flow Shapes

Shape	Purpose	Creates Assignment?
Assignment	Human task	Yes
Decision	Branch on When rule	No
Utility	Automated step (DT/Activity)	No
Subprocess	Call another flow	Depends
Wait	Pause for event/timer	Yes (pending)
Split-Join	Parallel branches, then join	Per branch
Split-ForEach	Loop over page list	Per iteration
Notify	Send notification	No

Q: Flow vs Flow Action?

Flow — backend process (shapes). **Flow Action** — user-facing button (Approve, Submit) with UI section. Flow Action may call a flow or just update assignment.

Q: Local vs Connector Flow Action?

Local — stays on same screen (modal/inline). **Connector** — navigates to new assignment/harness.

Chapter 7: Assignments & Routing

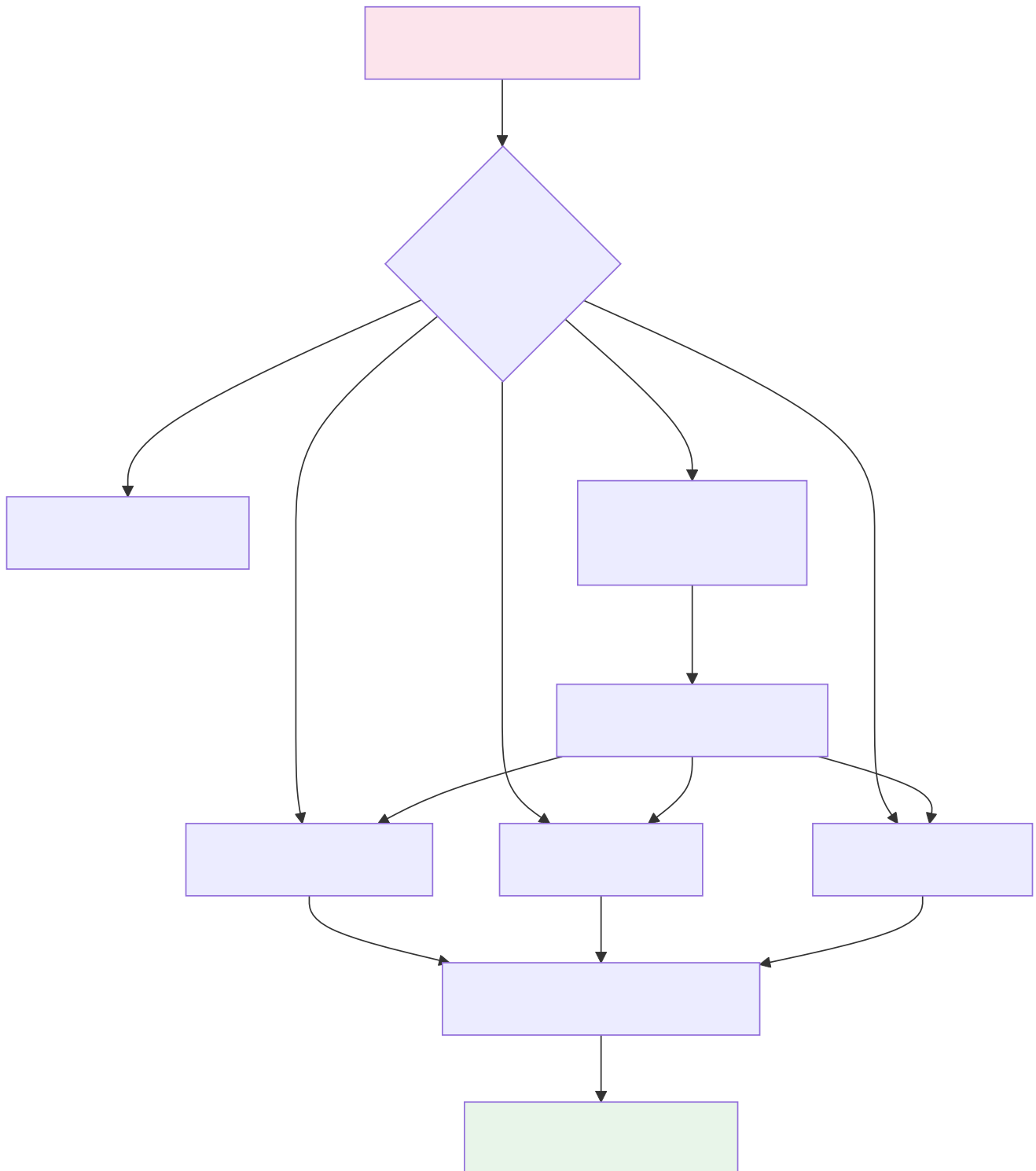


Figure: Assignment Routing Decision Flow

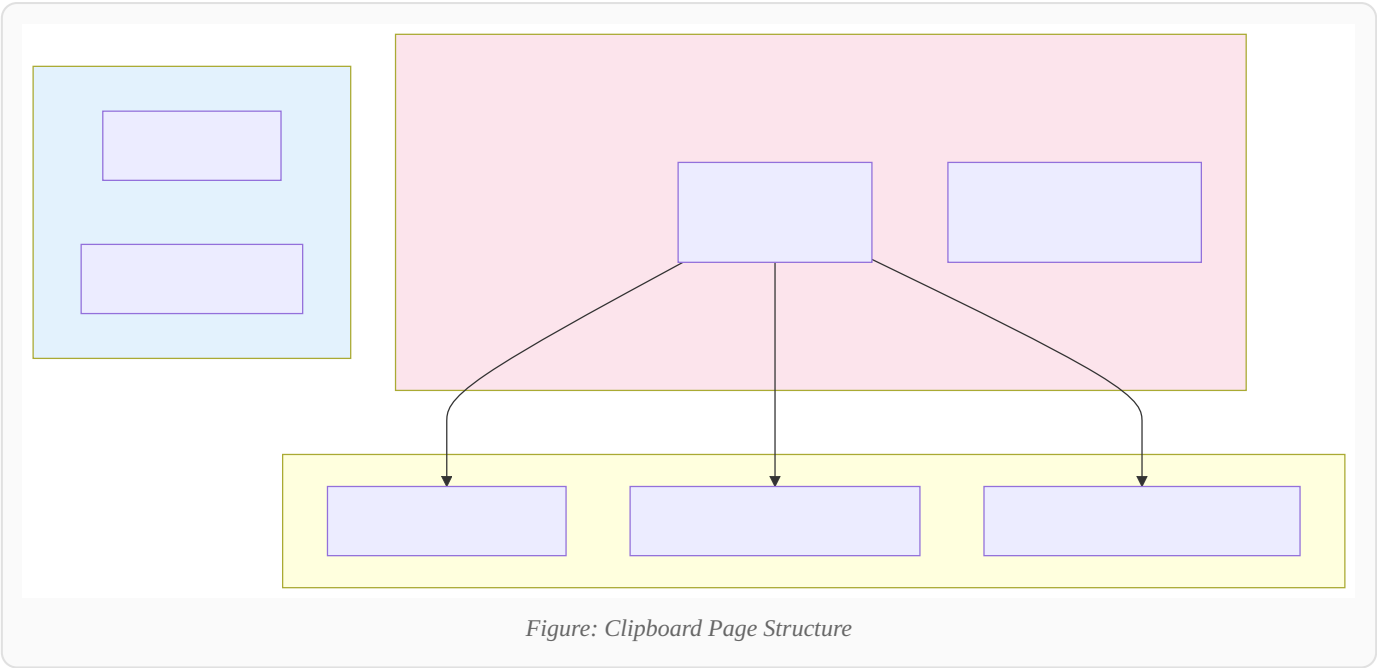
Q: Worklist vs Workbasket vs Work Queue?

Worklist — personal queue for one operator. **Workbasket** — shared team pool. **Work Queue** — named queue, can be routed dynamically.

Q: What is urgency?

Numeric priority (10–100). Increases as SLA deadlines approach. Higher urgency = higher in worklist sort order.

Chapter 8: Clipboard & Data Model



Property Types — Complete

Type	Description	Syntax
Text	String	<code>.Name</code>
Integer / Decimal	Numbers	<code>.Amount</code>
Date / DateTime	Temporal	<code>.SubmittedDate</code>
TrueFalse	Boolean	<code>.IsActive</code>
Page	Embedded object	<code>.Customer</code>
Page List	Ordered collection	<code>.Items(1)</code>
Page Group	Keyed collection	<code>.Docs("ID1")</code>
Value List / Group	Scalar collections	<code>.Tags(1)</code>

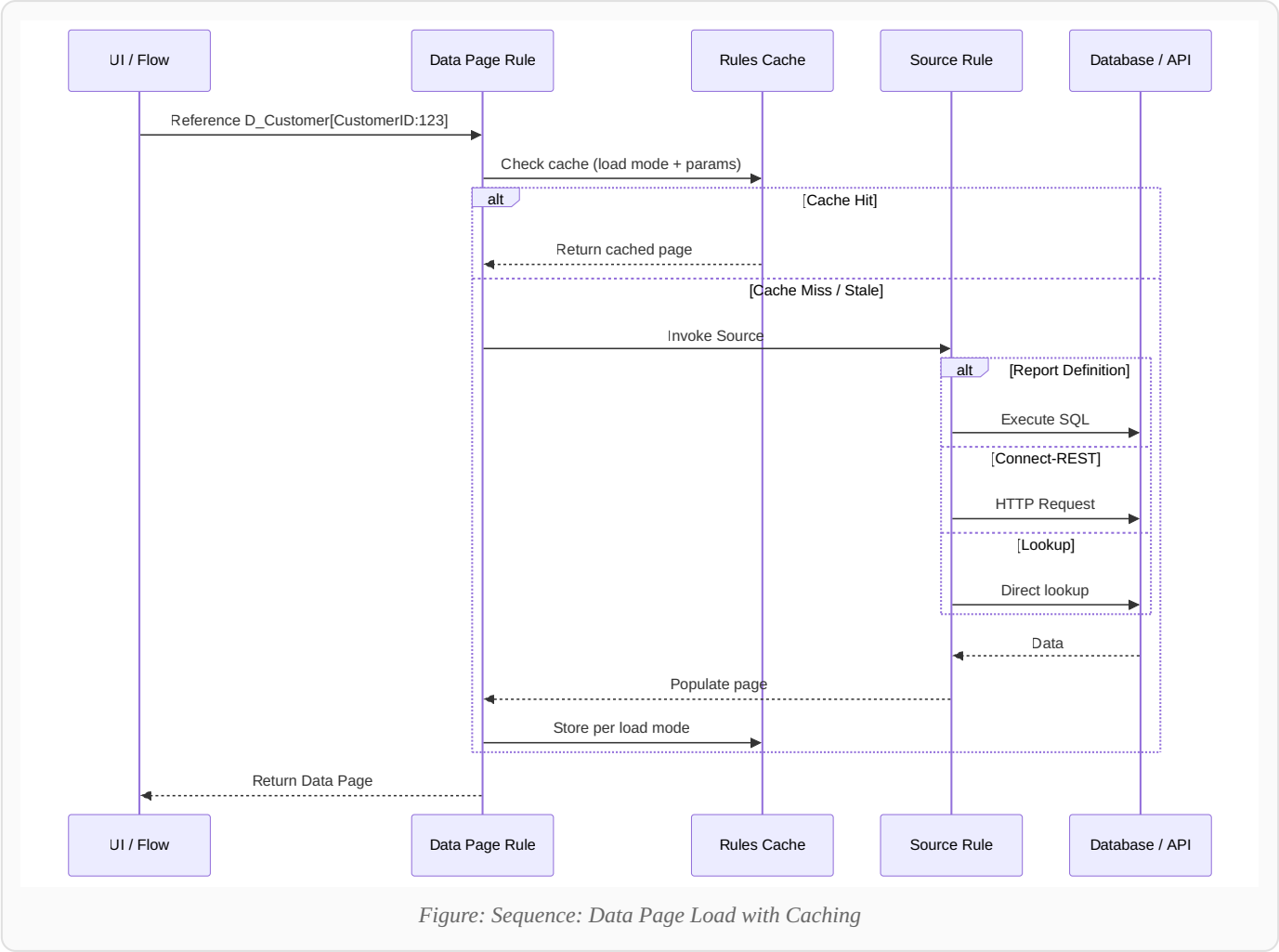
Q: Page List vs Page Group?

Page List — integer-indexed, ordered. **Page Group** — string-keyed, unordered. Use list for sequences, group for named lookups.

Q: Embedded vs Linked pages?

Embedded — stored inside parent (serialized together). **Linked** — reference via `pxLinkedRefTo` ; separate clipboard page.

Chapter 9: Data Pages



Load Modes

Mode	Scope	Use When
Reload per interaction	Request	Always fresh data
Reload once per requestor	User session	User-specific reference data
Reload if older than X	Time-based	Semi-static data
Access group	Access group	Shared config per app role
Node	Cluster node	Expensive, rarely changing data

Q: Data Page sources?

Report Definition, Activity, Connector (REST), Lookup (DB), Data Transform, or another Data Page.

Q: Savable Data Page?

Editable Data Page with **Save Plan** (Database/Activity) to persist changes. Used in Constellation for inline CRUD on Data Types.

Chapter 10: Data Transforms

All DT Actions

Set · Update Page · Append and Map to · Remove · When / Otherwise · For Each Page In · Apply Data Transform · Sort · Exit Data Transform

Q: DT vs Activity?

Always prefer **Data Transform** for mapping/simple logic. Use **Activity** only for complex iteration, integration orchestration, or legacy maintenance.

Chapter 11: Activities

Key Methods

Method	Purpose
Property-Set	Set property value
Call	Invoke sub-activity
Apply-DataTransform	Run DT
Obj-Open / Obj-Save / Obj-Delete	Persistence
Page-New / Page-Remove / Page-Copy	Clipboard management
RDB-List / RDB-Open	Direct SQL (avoid if possible)
Connect-REST	Call REST connector
Queue-For-Agent	Async processing

Guardrail: Activities are procedural and being phased out. Say "DT first, Activity only when necessary."

Chapter 12: Decision Rules

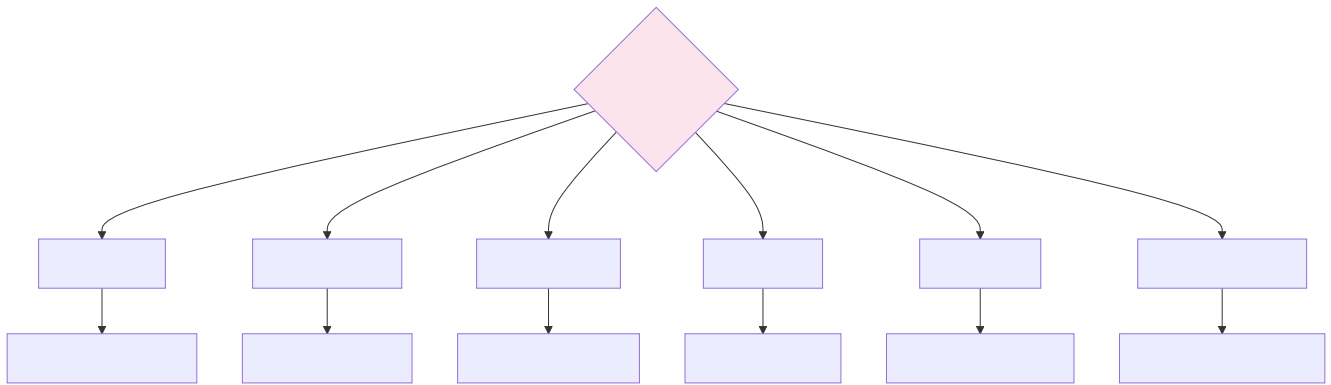


Figure: Decision Rule Selection Flowchart

Q: Decision Table vs Tree?

Table — grid of AND/OR conditions → outcomes; best for combinatorial logic. **Tree** — sequential if-else hierarchy; best for nested decisions.

Q: What is a Decision Strategy?

Component of **Customer Decision Hub (CDH)**. Combines Propositions, Filtering, Prioritization, and Arbitration for Next-Best-Action across channels.

Chapter 13: Declare Rules

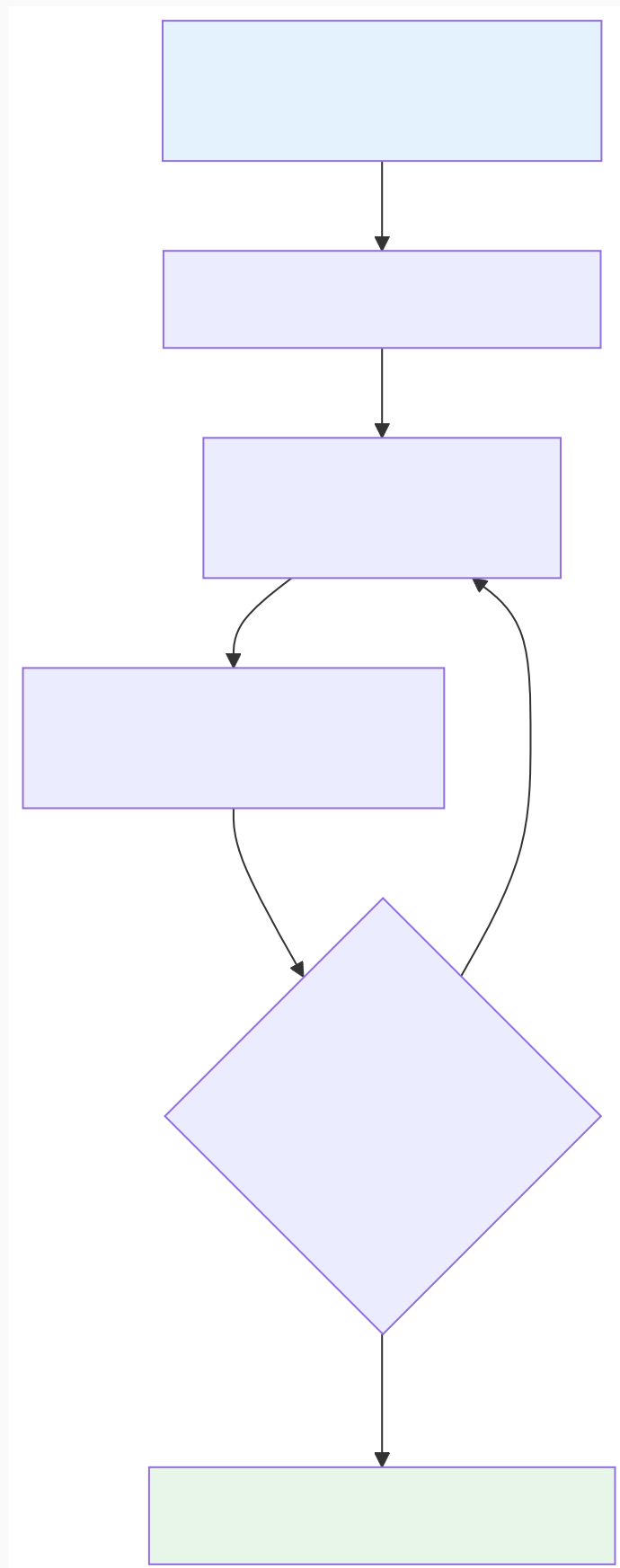


Figure: Declare Expression — Forward Chaining

Rule	Fires When	Purpose
Declare Expression	Source property changes	Computed property (forward chaining)
Declare Constraint	Property set/save	Validation constraint
Declare OnChange	Property value changes	Trigger DT/Activity
Declare Trigger	Data instance committed	Post-save processing on Data- class
Declare Index	Instance saved	Maintain search index

Chapter 14: Validation Rules

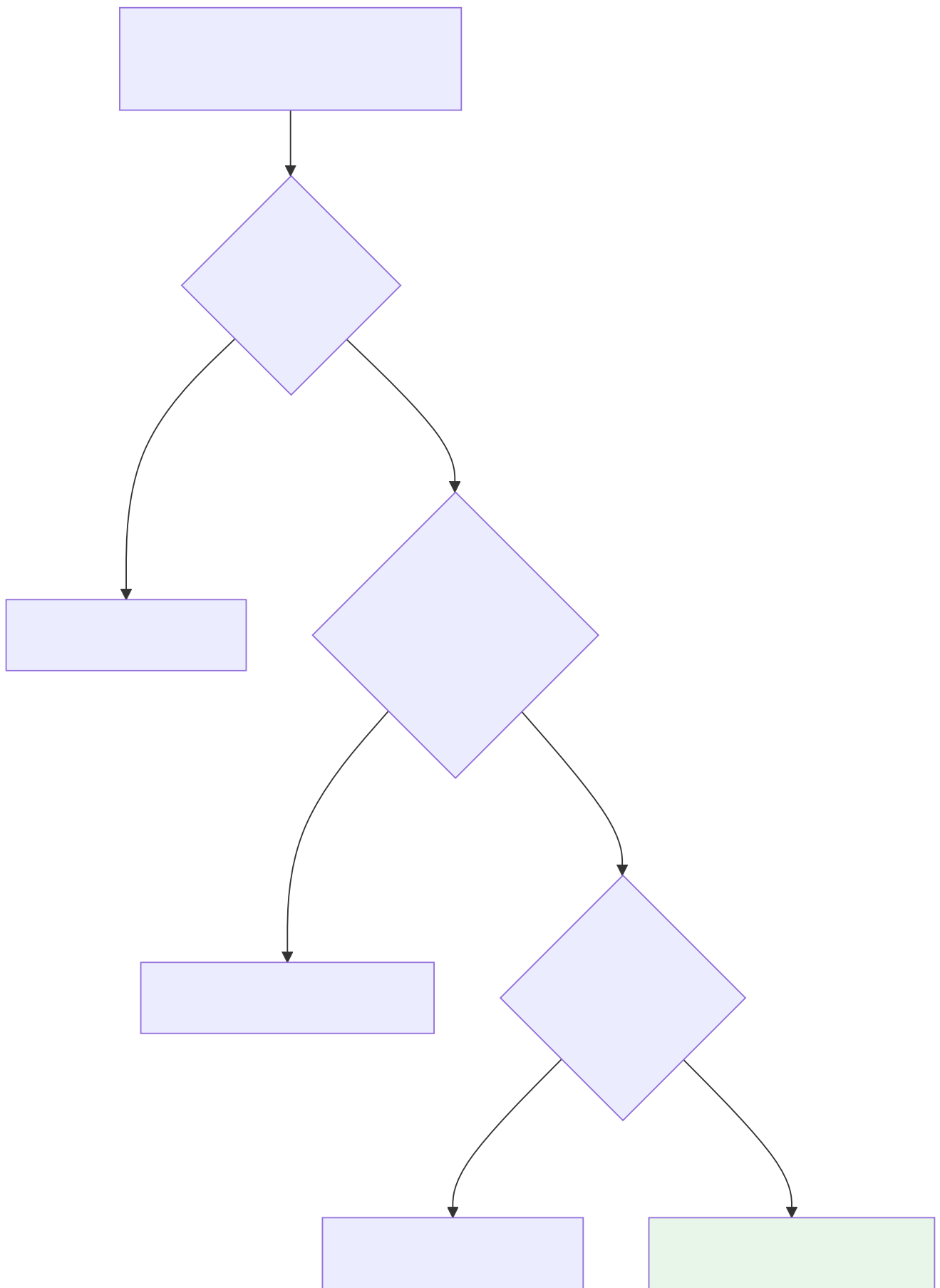
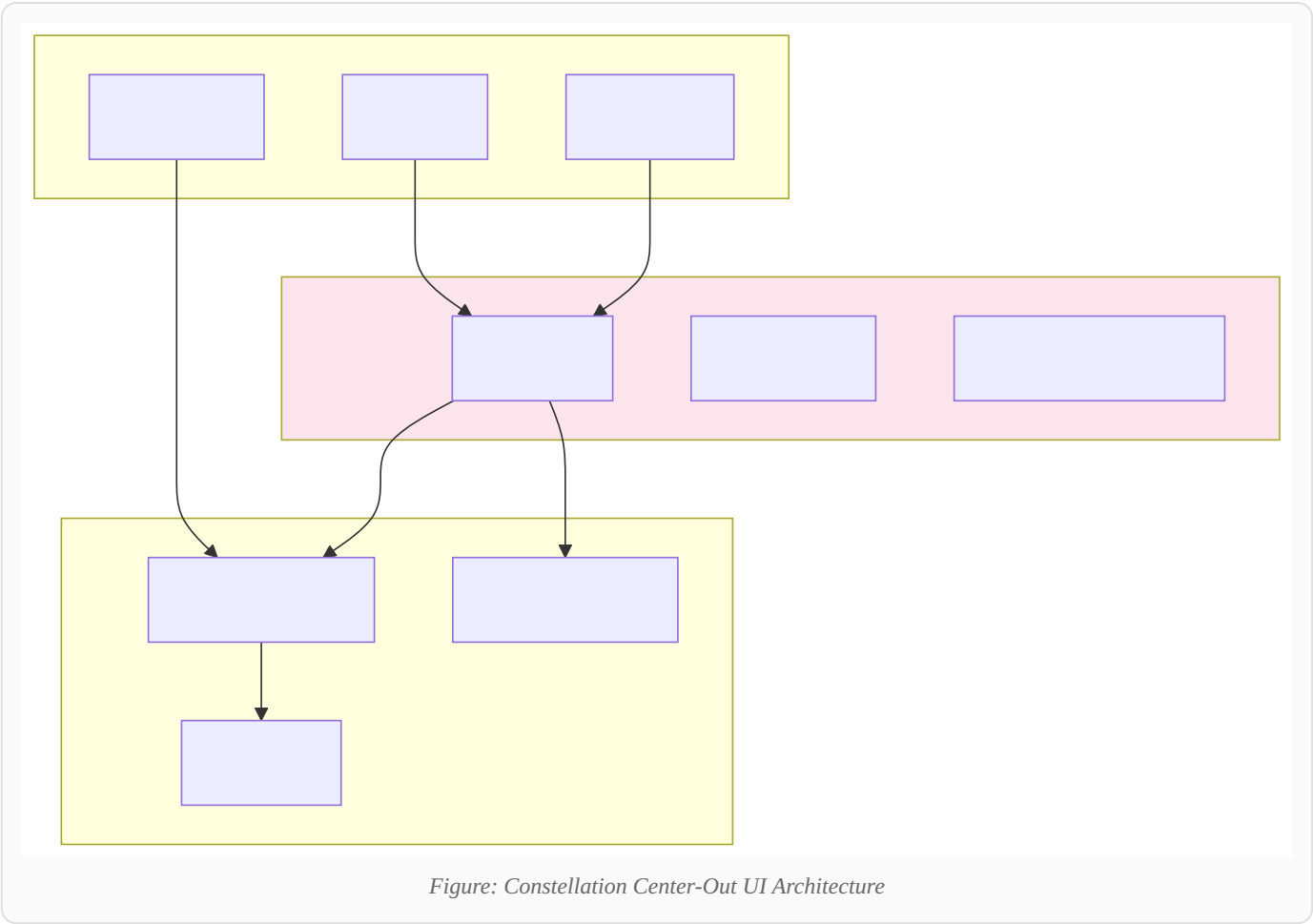


Figure: Validation Flow — Edit Validate → Validate → Constraint → Save

Rule	When	Where
Validate	Server-side on commit	Flow, Activity, DT
Edit Validate	Client + server	Property control, section
Declare Constraint	Property change	Declarative, any time

Chapter 15: UI — Traditional & Constellation



Traditional	Constellation (Modern)
Section	View
Harness	Full Page / Form
Portal	Shell + Theme
Dynamic Layout	Cosmos React components
Skin	Theme (design tokens)

Q: Why Constellation?

Responsive, omnichannel, React-based, faster development, no heavy harness customization, better performance, channel-agnostic via DX API.

Chapter 16: SLAs & Urgency

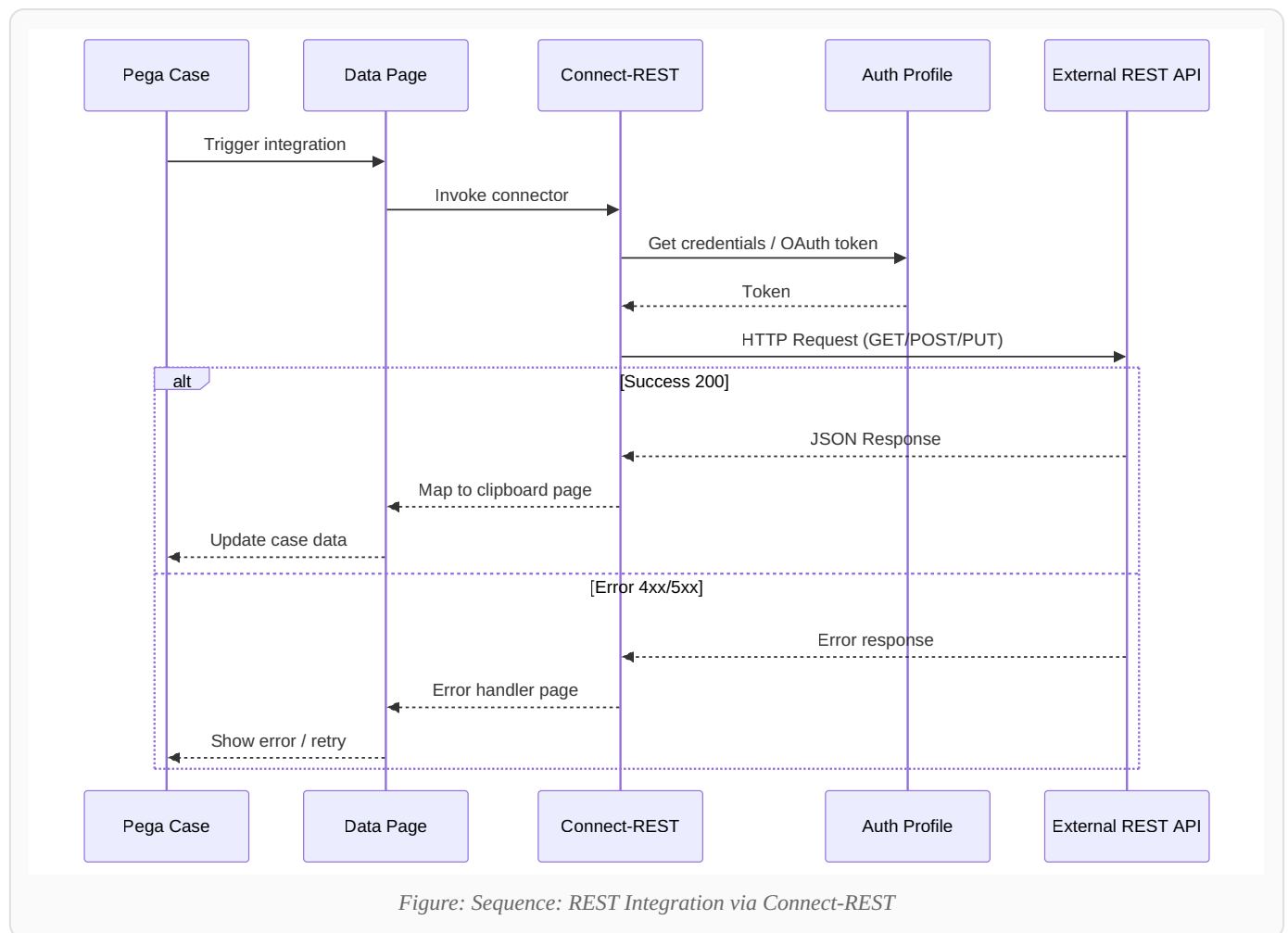


Figure: SLA Timeline — Goal → Deadline → Passed Deadline

Q: Assignment SLA vs Case SLA?

Assignment SLA — per task. **Stage/Case SLA** — per stage or overall case duration.

Chapter 17: Integration & Data Types



Rule	Direction
Connect-REST / SOAP / SQL / Kafka / MQ	Outbound
Service-REST / SOAP / Kafka	Inbound

Q: What is a Data Type?

Pega-managed Data - class with auto-generated CRUD UI, REST API, and storage. Modern way to manage master/reference data.

Q: Authentication options?

Basic Auth, OAuth 2.0, JWT, API Key, Custom (Authentication Service rule). Outbound uses Authentication Profile.

Chapter 18: Security Model

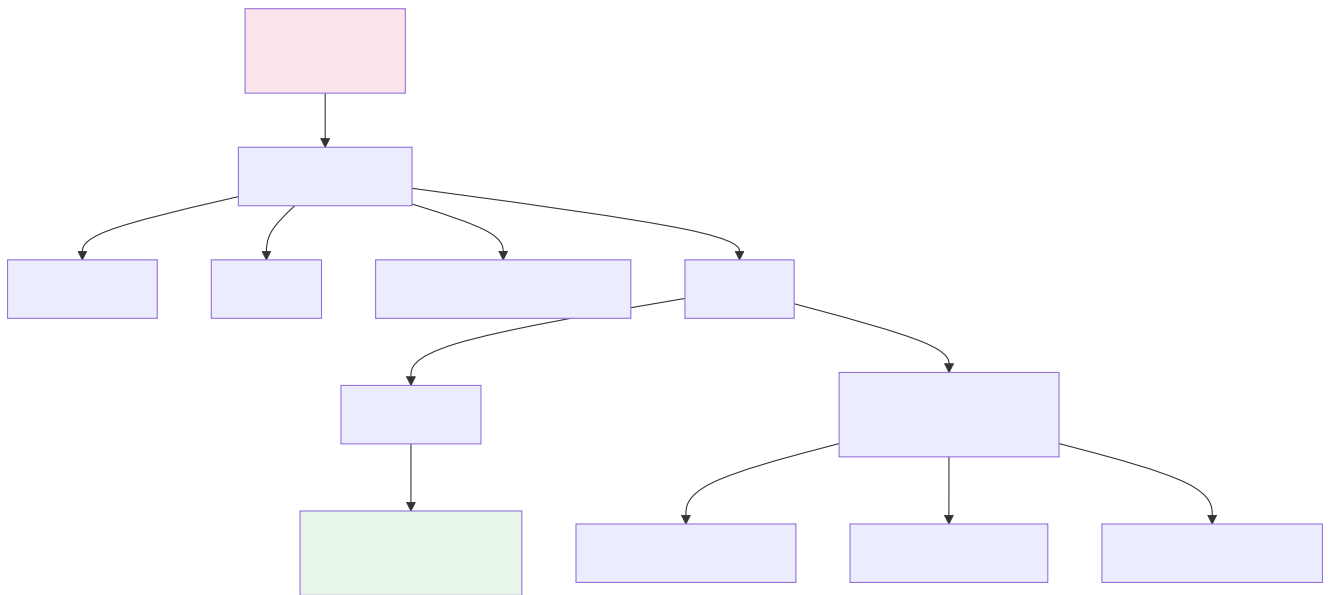
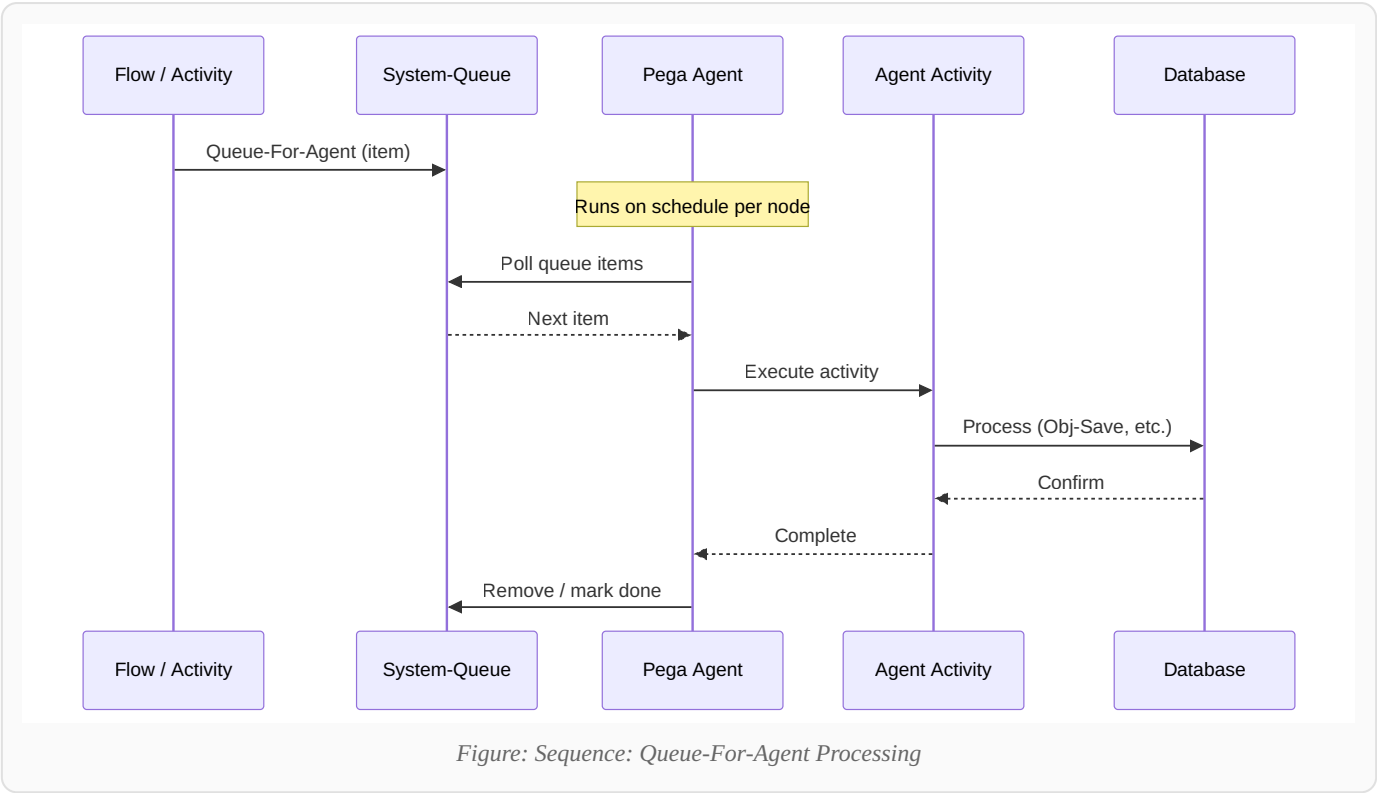


Figure: Security Chain — Operator to Privilege

Q: What is ABAC?

Attribute-Based Access Control — access based on property values (region, department). Implemented via Access When rules on cases, properties, flow actions.

Chapter 19: Agents, Queues & Job Schedulers

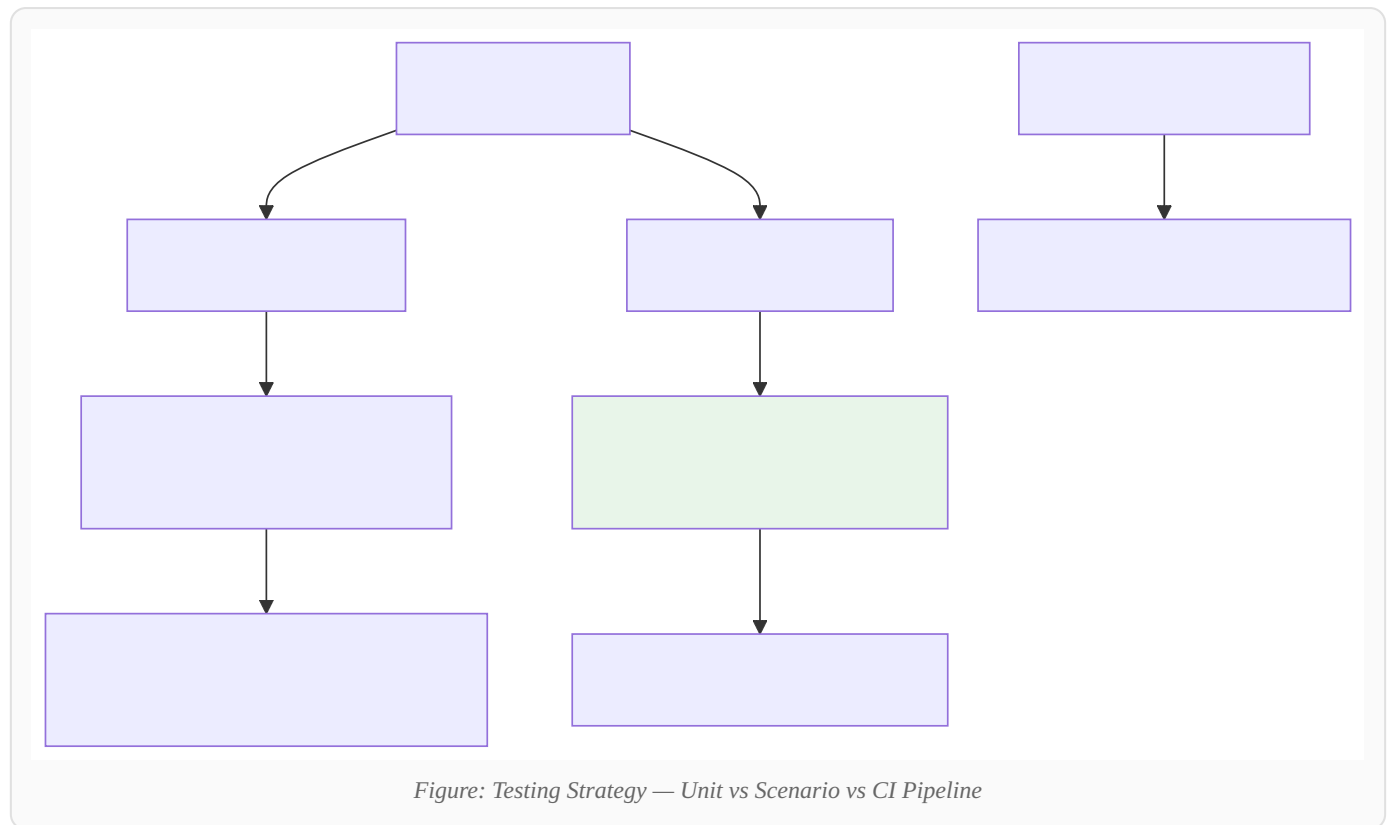


	Agent	Job Scheduler
Best for	High-volume async queues	Scheduled tasks
Modern preference	Legacy for new work	Preferred for new dev

Chapter 20: Reporting

Report Definition replaces Summary View. Uses SQL via association rules for joins. Optimize with indexes, limit columns, avoid functions on indexed WHERE columns.

Chapter 21: Testing Strategy

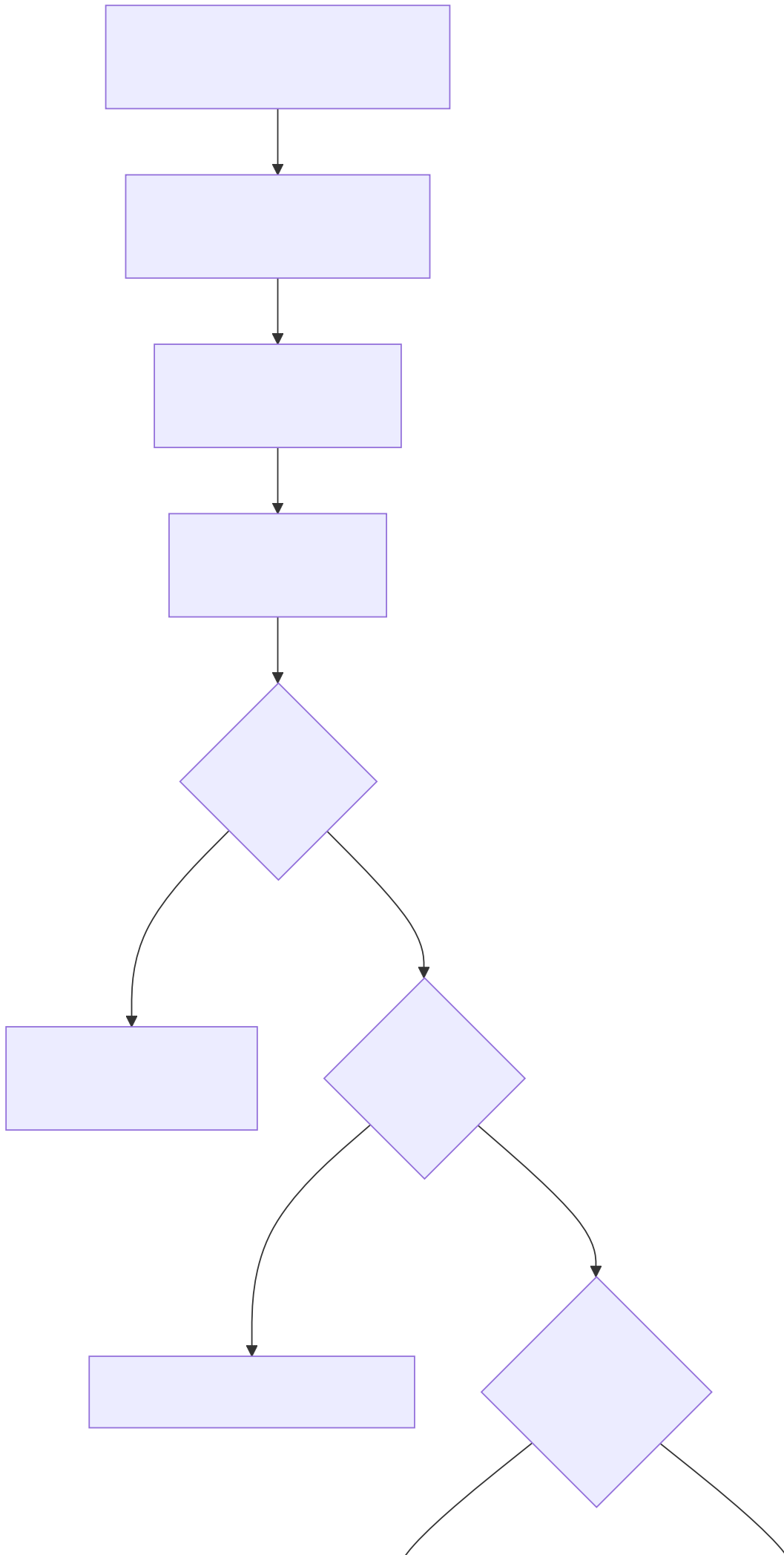


Unit Test Case — individual rules (DT, Activity, Decision). **Scenario Test** — end-to-end case lifecycle (UI automation). Use dedicated **Test Application** and test ruleset.

Q: How to run tests in CI?

Export test cases in RAP, import to QA via Deployment Manager pipeline, execute PegaUnits + scenario tests as pipeline stage gate before production promotion.

Chapter 22: Performance, PAL & Guardrails



Performance Tools

PAL · Tracer · Clipboard Inspector · DB Trace · Rule Inspector

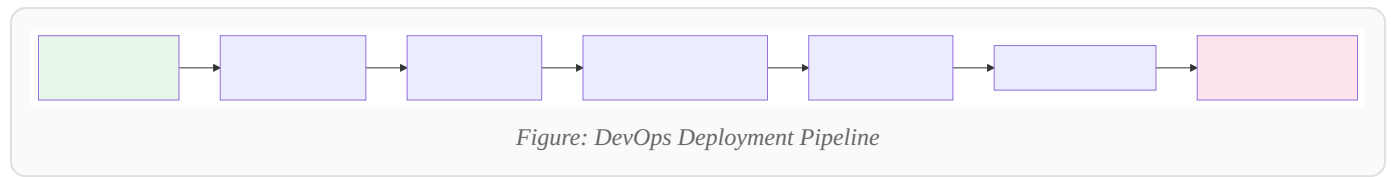
Complete Guardrails List

1. Max 5 flow actions per assignment
2. Prefer Data Transforms over Activities
3. Limit ~200 properties per class
4. Use Data Pages, not repeated DB calls
5. No looping Activities on large datasets
6. Use Declare Index, not RDB-List in loops
7. Don't store large blobs in clipboard
8. Prefer Report Definition over RDB-List
9. Use pagination for large lists
10. Avoid custom Java when OOTB works
11. Use dynamic layouts, not freeform
12. Limit cascade of sub-activities
13. Keep clipboard under 2MB per request
14. PAL: alert if DB ops > 50 or clipboard > 2MB

Q: N+1 query problem?

Loop calling Obj-Open/Data Page per row = N+1 DB calls. Fix: batch Data Page, Report Def with join, paginated load.

Chapter 23: DevOps & Deployment



RAP = Ruleset Archive Package. **Product Rule** defines contents. **Deployment Manager** orchestrates pipelines. **Git Repository** for version control. **Skim** removes unreferenced rules before packaging.

Chapter 24: Email, Documents & Correspondence

Correspondence rule — email templates. **Send Email Smart Shape** in flows. **Email Account** — SMTP config. **Inbound email** — Service Email creates/updates cases. **Document generation** — HTML/PDF templates attached via correspondence.

Chapter 25: Advanced — CDH, RPA, Data Flow, AI

CDH — Next-Best-Action via Strategies. **RPA** — Robot Studio automates desktop apps. **Data Flow** — streaming/batch pipeline (Kafka). **Prediction Studio** — ML models. **GenAI** — case summarization, email drafting, Socrates chat.

Chapter 26: Controls, Field Values & Localization

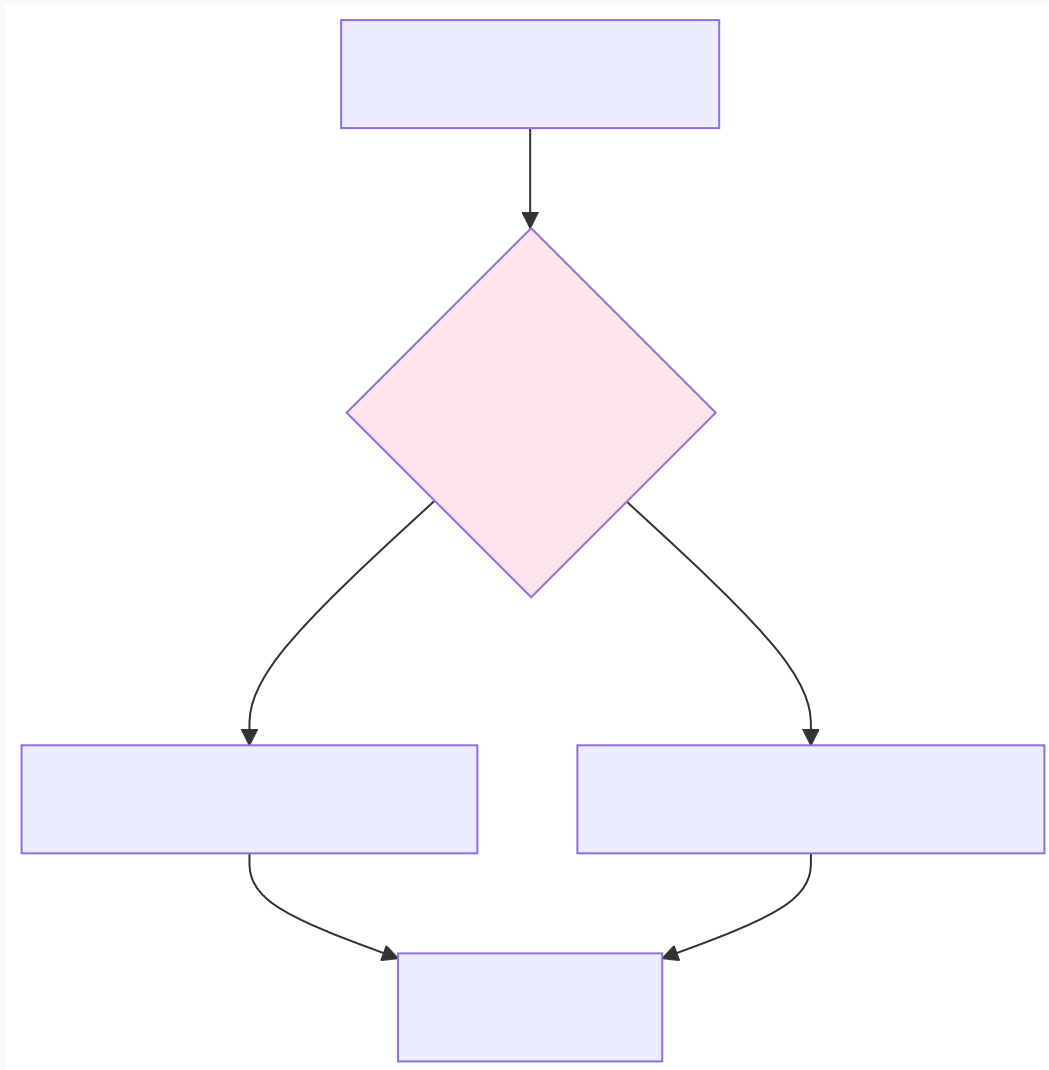


Figure: Feature Toggle — Runtime Feature Switching

Field Value — key-value pairs for dropdowns. **Localization** — locale-specific labels via Field Value rules. **Feature Toggle** — runtime feature on/off without deployment. Controls: pxDropdown, pxAutoComplete, pxDateTime, pxTextInput.

Chapter 27: Object Layer & Persistence

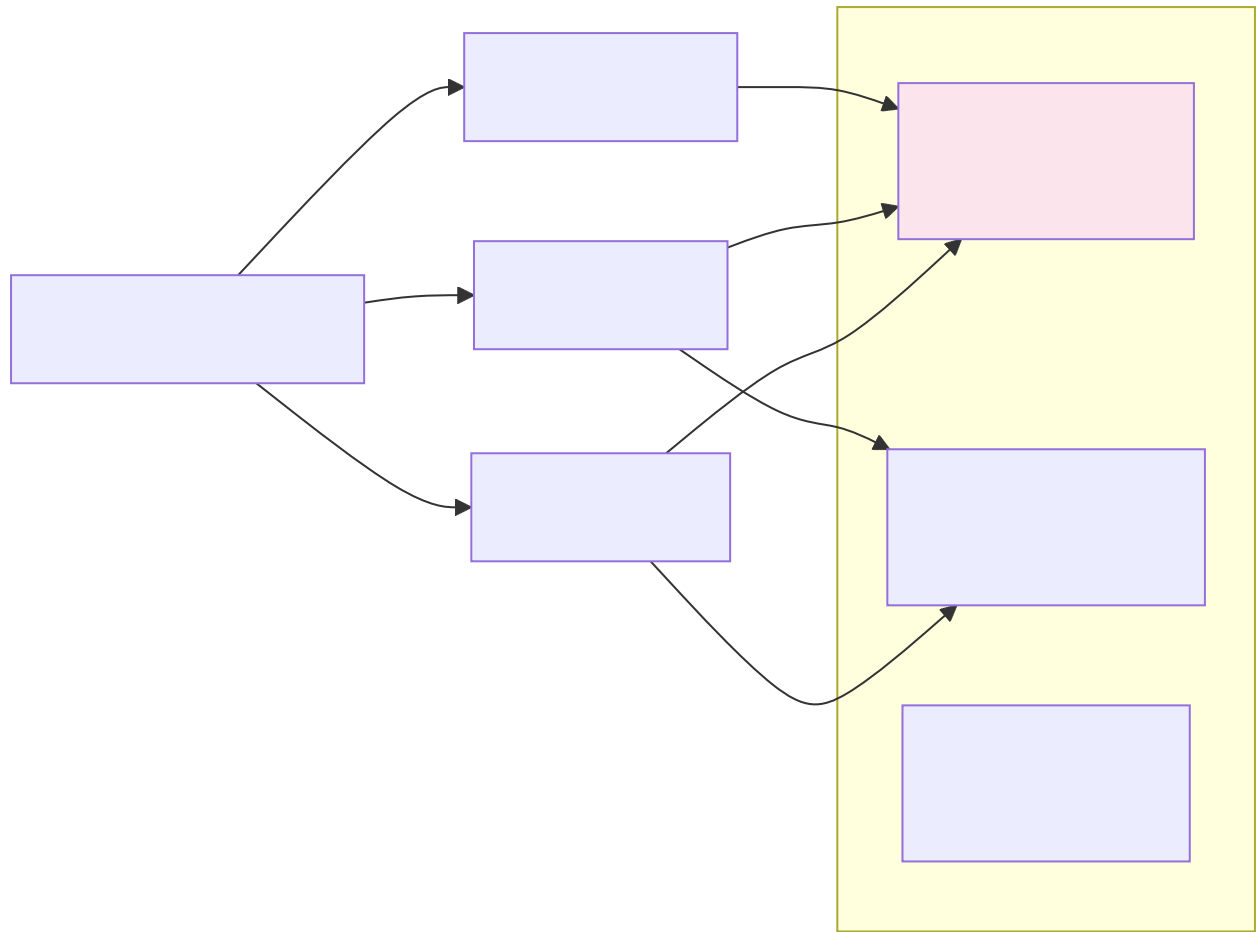


Figure: Object Persistence — Obj-Open / Obj-Save / Obj-Delete

Obj-Open — read from DB. **Obj-Save** — write to DB. **Obj-Delete** — remove. **Obj-Open-By-Handle** — open by pzInsKey. **pxCommitDateTime** — last DB commit. **pzInsKey** — unique instance key.

Q: Obj-Save vs Commit?

Obj-Save writes instance to DB. In flows, the engine auto-commits on assignment submit. In activities, may need **Commit** method for transaction control.

Chapter 28: 200 Rapid-Fire Questions

200 questions. Zero escape. You survive this chapter, you survive any panel.

1. BPM? → Business Process Management
2. Case class prefix? → Work-
3. Default flow? → pyDefault
4. Case creator property? → pxCreateOperator
5. Case status? → pyStatusWork
6. Assignment urgency? → pxUrgencyAssign
7. Last update time? → pxUpdateDateTime
8. Rule base class? → Rule-
9. Ruleset version format? → MM-mm-nn
10. FUA? → Full Rule Assembly cache
11. Work pool? → Class group
12. Commit time? → pxCommitDateTime
13. Assignment class? → Assign-
14. Current flow? → pyFlowName
15. Step page? → pyStepPage
16. Current stage label? → pxCurrentStageLabel
17. Resolve? → Complete assignment
18. Spin-off? → Create child case
19. Cover case? → Parent of related cases
20. Cover key? → pxCoverInsKey
21. DCO? → Direct Capture of Objectives
22. Pega Express? → Rapid delivery methodology
23. Guardrails score? → Application health %
24. PAL DB warning? → >50 operations
25. Clipboard warning? → >2MB
26. DB ops counter? → pxRDBIO
27. Field Value? → Dropdown key-value
28. Validate rule? → Server validation
29. Edit Validate? → Client+server validation
30. Constraint? → Declare Constraint
31. Route to? → pxRouteTo
32. WorkBasket? → Team queue
33. Org unit? → pxOrgUnit
34. Division? → pxDivision
35. Application name? → pxApplication
36. Process name? → pxProcessName
37. Task name? → pxTaskName
38. Constellation? → Modern React UI
39. Cosmos? → Design system
40. DX API? → Digital Experience API
41. pxAPI? → Pega REST layer
42. Instance key? → pzInsKey
43. Object class? → pxObjClass
44. Open by handle? → Obj-Open-By-Handle
45. Linked page ref? → pxLinkedRefTo
46. Access role rule? → Rule-Access-Role-Obj
47. Access When? → Rule-Access-When
48. Thread name? → pxThreadName
49. Page list index? → pxSubscript
50. Results page? → pxResults
51. RDB-List? → SQL list in activity
52. Obj-Sort? → Sort page list
53. Obj-Filter? → Filter page list
54. Forward chaining? → Auto recalc declares
55. Data Page params? → pxDPPParameters
56. SLA queue class? → System-Queue-ServiceLevel
57. Core ruleset? → Pega-Engine
58. Process ruleset? → Pega-ProCom
59. Root class? → @baseclass
60. Withdrawn? → Deprecated rule
61. Final rule? → Cannot override
62. Blocked? → Excluded from resolution
63. Available? → Active rule
64. Not Available? → Saved but inactive
65. Current stage? → pxCurrentStage
66. Sub-status? → pxSubStatus
67. Save time? → pxSaveDateTime
68. Update operator? → pxUpdateOperator
69. Case links? → pyCaseLinks
70. Work parties? → pyWorkParties
71. Party role? → pxPartyRole
72. MapValue use? → 1:1 lookup mapping
73. Scorecard use? → Weighted scoring
74. Strategy use? → CDH decisioning
75. Split-Join? → Parallel processing
76. Split-ForEach? → Loop page list in flow
77. Utility shape? → Automated no-assignment step
78. Subprocess? → Call another flow
79. Wait types? → Timer, case dependency, event
80. Reload once? → Data page load mode
81. Node scope cache? → Shared per cluster node
82. Save Plan? → Persist editable data page
83. Authentication Profile? → Outbound auth config
84. Service Package? → Groups inbound services
85. Product rule? → Defines RAP contents
86. Skim? → Remove unreferenced rules

- 87. Branch merge? → Combine branch to target
- 88. Scenario test? → E2E UI automation
- 89. Unit test? → Single rule test
- 90. Tracer? → Real-time rule execution trace
- 91. PAL? → Performance Analyzer
- 92. Declare Index? → Search index maintenance
- 93. Declare Trigger? → On commit of Data- instance
- 94. Declare OnChange? → On property change
- 95. Proposition? → CDH offer/action
- 96. Adaptive model? → Self-learning ML
- 97. Text analyzer? → NLP in Prediction Studio
- 98. Robotic automation? → Desktop app automation
- 99. Job Scheduler vs Agent? → Scheduled vs queue polling
- 100. Queue-For-Agent? → Defer to agent queue
- 101. Association rule? → Join classes in reports
- 102. Report Definition? → Modern SQL report rule
- 103. Summary View? → Legacy (use Report Def)
- 104. Harness types? → New, Perform, Review, Confirm
- 105. Local flow action? → Same screen action
- 106. Connector flow action? → Navigates to new screen
- 107. Dynamic layout? → Responsive flex layout
- 108. Freeform layout? → Legacy absolute positioning
- 109. Skin rule? → Application look and feel
- 110. Portal? → Application shell
- 111. Work group? → Group of operators
- 112. Access group? → App + roles + portal
- 113. Privilege? → Atomic permission
- 114. Role? → Collection of privileges
- 115. ABAC? → Attribute-based access control
- 116. Property security? → Read/write restrictions
- 117. Circumstance date? → Time-based rule variant
- 118. Circumstance template? → Property-based variant
- 119. Directed inheritance? → Explicit parent class
- 120. Pattern inheritance? → Namespace hierarchy
- 121. Class group table? → pc_work (default)
- 122. Data type REST? → Auto-generated CRUD API
- 123. Connect-Kafka? → Outbound event streaming
- 124. Data Flow? → Batch/stream processing pipeline
- 125. Data Set? → Source/sink for data flows
- 126. Feature toggle? → Runtime on/off switch
- 127. Localization? → Locale-specific labels
- 128. Multi-tenancy? → Logical tenant isolation
- 129. Repository? → Git integration for rules
- 130. Deployment pipeline? → Dev → QA → Prod automation
- 131. Guardrail: flow actions? → Max 5 per assignment
- 132. Guardrail: activities? → Avoid; use DT
- 133. Guardrail: properties? → ~200 per class
- 134. pyWorkPage? → Primary case data page
- 135. pxRequestor? → Operator session context
- 136. pxThread? → Session thread with assignments
- 137. param page? → Activity/DT parameters
- 138. Step page? → Flow shape page context
- 139. Page-Copy? → Copy clipboard page
- 140. Page-Merge? → Merge two pages
- 141. Property-Map? → Map between pages
- 142. Show-Page? → Display harness
- 143. Log-Message? → Write to log
- 144. Rule resolution cache? → FUA
- 145. Application rule? → Defines app stack
- 146. Built-on application? → Parent app inheritance
- 147. Instance handle? → pxInsHandle
- 148. Operator ID page? → OperatorID
- 149. Workbasket vs worklist? → Team pool vs personal
- 150. Goal interval? → SLA first threshold
- 151. Deadline interval? → SLA must-complete
- 152. Passed deadline? → SLA overdue actions
- 153. SLA agent? → Pega-ProCom:ServiceLevelEvents
- 154. Urgency initial? → Default 10
- 155. Urgency max? → Typically 100
- 156. GenAI Socrates? → AI assistant in App Studio
- 157. Center-out? → Design once, deploy everywhere
- 158. Pega Infinity? → Unified 8.x+ platform
- 159. App Studio persona? → Business developer UX
- 160. Admin Studio? → System administration
- 161. Prediction Studio? → ML model management
- 162. DCR? → Direct Capture Requirements
- 163. View (Constellation)? → Replaces Section
- 164. Widget? → Reusable UI component
- 165. Theme? → Constellation design tokens
- 166. Editable data page? → Supports save plan
- 167. Read-only data page? → Lookup/reference
- 168. List data page? → Returns page list
- 169. Singleton data page? → Single object
- 170. Stale-while-revalidate? → Return cache, refresh async
- 171. HTTP 401 handling? → Auth error in connector
- 172. OAuth 2.0? → Token-based auth
- 173. JWT? → JSON Web Token auth
- 174. Keystore? → Certificate storage
- 175. Inbound email service? → Create case from email
- 176. Correspondence? → Email/letter template
- 177. PDF generation? → HTML template to PDF
- 178. AttachAsPDF? → Attach doc to case
- 179. RPA robot? → Desktop automation bot
- 180. Robot Studio? → Build RPA automations
- 181. NBAM? → Next Best Action Marketing
- 182. Proposition filter? → Filter offers in strategy
- 183. Prioritization? → Rank propositions
- 184. Arbitration? → Resolve strategy conflicts

185. Simulate? → Test decision strategies
186. Champion-challenger? → A/B test strategies
187. Text extraction? → NLP from documents
188. Sentiment analysis? → NLP feature
189. Entity extraction? → NLP feature
190. pxAPI case create? → REST case creation
191. pxUpdateCase? → REST case update
192. DX API v2? → Constellation API layer
193. Dynamic system setting? → Config via DSS rule
194. Declare Expression target? → Computed property
195. Declare Expression source? → Dependency properties
196. Auto-populate? → Property triggers DT/Activity

197. Edit input? → Client-side validation
198. Constraint message? → Error on violation
199. When in DT? → Conditional step
200. For Each Page In? → Loop in DT
201. Exit DT? → Stop transform execution
202. Call activity? → Invoke sub-activity
203. Branch in activity? → Conditional jump to label
204. Activity allow list? → Security restriction
205. Restricted activity? → Cannot call from UI
206. Obj-Delete? → Remove DB instance
207. History-Work? → Case audit trail
208. Index-Work? → Search index class

Chapter 29: Mock Interview Scenarios

Scenario 1: Design a Loan Application case type.

Stages: Application → Underwriting → Approval → Fulfillment. Data: Applicant, Income, CreditScore. Integrations: Credit Bureau via Connect-REST + Data Page. Decisions: Decision Table for eligibility. SLAs: 48hr underwriting deadline. Security: Loan Officer vs Underwriter roles.

Scenario 2: Production case is slow. How do you debug?

PAL before/after → Tracer with filters → DB Trace for SQL → Check Data Page load modes → Look for looping activities → Review clipboard size → Check N+1 patterns → Review guardrails compliance.

Scenario 3: Migrate traditional UI to Constellation.

Enable Constellation on case type → Convert Sections to Views → Replace Harnesses with Full Pages → Update Theme → Test DX API → Validate all flow actions → Run scenario tests → Deploy via pipeline.

Scenario 4: Implement role-based case access by region.

Create Access When rules checking `.Region = OperatorID.pyRegion`. Apply to case type, list views, and flow actions. Configure ABAC on property security for sensitive fields.

Chapter 30: Final Checklist & STAR Stories

You made it — all 30 chapters, 18 diagrams, 200 rapid-fire questions. You're not just prepared; you're dangerous. Go get that offer.

Day-Before Checklist

- ☒ Draw Rule Resolution flowchart from memory
- ☒ Draw Case Lifecycle state diagram
- ☒ Explain Data Page load sequence
- ☒ Walk through REST integration flow
- ☒ Recite Security chain
- ☒ Name all flow shapes
- ☒ Compare DT vs Activity vs Data Page
- ☒ Explain Constellation vs Traditional
- ☒ Describe deployment pipeline
- ☒ List 10 guardrails
- ☒ Prepare 3 STAR project stories

Every concept. Every diagram. Every question. Nothing missed. Now go be brilliant.